



KTH ROYAL INSTITUTE OF TECHNOLOGY

---

# **A Numerical Comparison of Two Discrete Models of Surface Bending**

*Curvature based Bending Forces on Triangulated Meshes*

---

Joyjeet Dutta, Dhrubaditya Mitra

Digital Futures Summer Internship 2026

# Contents

|          |                                           |           |
|----------|-------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                       | <b>3</b>  |
| <b>2</b> | <b>Theory</b>                             | <b>3</b>  |
| 2.1      | Curvature Energy . . . . .                | 3         |
| 2.2      | Energy argument . . . . .                 | 4         |
| 2.3      | Flat Domain Hessian Energy . . . . .      | 4         |
| 2.4      | Isometric Bending Model . . . . .         | 7         |
| <b>3</b> | <b>Discretization</b>                     | <b>9</b>  |
| 3.1      | Flat Domain Hessian Energy . . . . .      | 9         |
| 3.1.1    | Time Complexity . . . . .                 | 11        |
| 3.2      | IBM . . . . .                             | 12        |
| 3.2.1    | Finite Element Discretization . . . . .   | 12        |
| 3.2.2    | Time Complexity . . . . .                 | 14        |
| <b>4</b> | <b>Test Case</b>                          | <b>14</b> |
| 4.1      | Flat Domain Hessian Energy . . . . .      | 14        |
| 4.2      | IBM . . . . .                             | 16        |
| <b>5</b> | <b>Method</b>                             | <b>20</b> |
| 5.1      | Mesh construction . . . . .               | 20        |
| 5.2      | Numerical Force Density . . . . .         | 20        |
| 5.3      | Mean Curvature Convergence . . . . .      | 21        |
| 5.4      | Time Benchmark . . . . .                  | 22        |
| <b>6</b> | <b>Results</b>                            | <b>22</b> |
| 6.1      | FDHE . . . . .                            | 22        |
| 6.2      | IBM . . . . .                             | 23        |
| 6.2.1    | Test Case . . . . .                       | 23        |
| 6.2.2    | Boundary Effects Test . . . . .           | 23        |
| 6.2.3    | Mean Curvature Convergence . . . . .      | 24        |
| 6.3      | Time Benchmark . . . . .                  | 26        |
| <b>7</b> | <b>Discussion</b>                         | <b>26</b> |
| 7.1      | Floating Point Round-off Errors . . . . . | 26        |
| 7.2      | Mean Curvature Convergence . . . . .      | 27        |
| 7.3      | Time complexity . . . . .                 | 27        |
| 7.4      | Applications . . . . .                    | 27        |

**8 Conclusion****28**

# 1 Introduction

Thin surfaces bend all around us, from sheets of paper to biological membranes reshaping within cells. Understanding this bending has applications ranging from cloth dynamics to simulations of exosomes. How can we model the forces that resist it? This resistance can be described by a curvature energy, whose variation gives the bending force density. In simulations, however, surfaces are represented by piecewise-flat triangular meshes, where curvature is not directly defined. This project compares two discrete bending models by examining their convergence under mesh refinement.

## 2 Theory

### 2.1 Curvature Energy

Let  $S$  be the two-dimensional surface embedded in three-dimensional Euclidean space. The surface is specified locally by the shape functions  $\mathbf{X} = (X^1, X^2, X^3)$  as

$$\mathbf{x} = \mathbf{X}(u^1, u^2)$$

where  $\mathbf{x} = (x^1, x^2, x^3)$  describe a point in space, and  $u^i$  are arbitrary *intrinsic* coordinates. We are interested in definitions of curvature, which come from the surface derivatives. The tangent vectors on the surface are given by the first surface derivatives

$$\mathbf{X}_\alpha = \partial_\alpha \mathbf{X} = \frac{\partial \mathbf{X}}{\partial u^\alpha}$$

The metric induced on the surface  $S$  by the embedding is defined by the inner product of the tangent vectors

$$g_{ij} = \langle \mathbf{X}_i, \mathbf{X}_j \rangle$$

The metric describes the intrinsic geometry of the surface: length scales and angles on the surface. The relevant notions of curvature on a 2D surface embedded in 3D is extrinsic. For this, we define the extrinsic curvature tensor, also known as the 2nd fundamental form,

$$b_{ab} = -(\mathbf{n} \cdot \partial_a \partial_b \mathbf{X})$$

The extrinsic curvature tensor measures the bending of the surface. The eigenvalues  $k_1, k_2$  of the extrinsic curvature tensor are the principal curvatures, and the trace of  $K$  is twice the mean curvature.

## 2.2 Energy argument

A lucid argument is presented for the bending energy functional. Note that energy is a scalar (zeroth order tensor). In order to relate the 2nd order extrinsic curvature tensor to a scalar (independent of coordinate system), we must find an invariant. The two invariants are the *trace*, and the *determinant* of the matrix  $b_{ij}$ . The determinant of  $b_{ij}$ , which is equal to the product of the eigenvalues, is the Gaussian curvature  $K = k_1 k_2$ . The trace is twice the mean curvature,  $2H = k_1 + k_2$ . The Gaussian curvature is an intrinsic property, depending only on the induced metric. We focus on the mean curvature,  $2H$ , as many applications, such as in soft matter, the Gaussian curvature term can be ignored in particular geometries (due to e.g. Gauss-Bonnet theorem). The next consideration to make is the direction of bending. The direction a surface is bent should not affect the sign of the bending energy, which motivates squaring the mean curvature in order to remove the sign. Lastly, since mean curvature is defined locally (at a point), we must sum together all the bending energy contributions (i.e. integrate over the surface). This leads to the bending energy functional (including a constant factor with the bending modulus  $\kappa$ )

$$E = \frac{\kappa}{2} \int_S (2H)^2 dA$$

A particularly important relationship is between the mean curvature and the Laplace-Beltrami operator on the surface,

$$\Delta_S \mathbf{X} = 2H \mathbf{n}$$

where  $\Delta_S$  is the Laplace-Beltrami operator (generalized Laplacian on the surface), and  $\mathbf{n}$  is the surface normal. This equation will be used for both numerical models

How can we derive a force from this? The force density is simply the variational derivative. The two numerical models build off of this energy functional, however different assumptions in the geometries reduce the problems and allow different simplifications.

## 2.3 Flat Domain Hessian Energy

The Flat Domain Hessian Energy is a smoothness energy for flat domains [2]. This model is applicable to a scalar function  $u$  defined on a *flat domain*,  $u : \Omega \in \mathbb{R}^2 \rightarrow \mathbb{R}$ . The bending energy functional is

$$E = \int_{\Omega} \nabla^2 u \, dA$$

How does this relate to a bending energy, and thereby force? First, consider a flat plane that is bent. Its height coordinate can be described by a scalar height field  $h(x, y)$ , with the embedding

$$\mathbf{X}(x, y) = \begin{pmatrix} x \\ y \\ h(x, y) \end{pmatrix}$$

The tangent vectors are  $\mathbf{X}_x = (1, 0, h_x)$  and  $\mathbf{X}_y = (0, 1, h_y)$  meaning that the metric is given by

$$g_{ij} = \begin{pmatrix} 1 + h_x^2 & h_x h_y \\ h_x h_y & 1 + h_y^2 \end{pmatrix}$$

$$\text{det}(g) = 1 + h_x^2 + h_y^2 = 1 + |\nabla h|^2$$

We can now write the Laplace-Beltrami operator in coordinates as

$$\Delta_g = \frac{1}{\sqrt{\text{det}g}} \partial_i (\text{det}g g^{ij} \partial_j)$$

Inserting the determinant of the metric, we get

$$\Delta_g \mathbf{X} = \frac{1}{\sqrt{1 + |\nabla h|^2}} \partial_i ((1 + |\nabla h|^2) g^{ij} \partial_j \mathbf{X})$$

We now note that the unit normal  $\mathbf{n} = \frac{(-h_x, -h_y, 1)}{\sqrt{1 + |\nabla h|^2}}$ . We can now insert all the terms in the curvature relationship

$$2H(-h_x, -h_y, 1) = \partial_i ((1 + |\nabla h|^2) g^{ij} \partial_j \mathbf{X})$$

In the  $e_3$  direction,

$$2H = \partial_i ((1 + |\nabla h|^2) g^{ij} \partial_j h)$$

$g^{ij}$  is the inverse of  $g_{ij}$

$$g^{ij} = \frac{1}{1 + |\nabla h|^2} \begin{pmatrix} 1 + h_y^2 & -h_x h_y \\ -h_x h_y & 1 + h_x^2 \end{pmatrix}$$

This gives

$$g^{ij}\partial_j h = \frac{\nabla h}{(1 + |\nabla h|^2)}$$

Lastly, taking the outer partial derivative gives the equation

$$2H = \nabla \cdot \left( \frac{\nabla h}{1 + |\nabla h|^2} \right) \quad (1)$$

From here we see that for *small deformations* i.e. where  $|\nabla u|^2 \sim 0$ , we get that  $2H \approx \Delta h$ , where  $\Delta$  is the cartesian Laplacian.

Under the small deformations assumption  $2H \approx \Delta h$ , and the bending energy functional is

$$E = \frac{\kappa}{2} \int (\Delta h)^2 dA$$

Importantly, the continuum energy involved the *hessian*: the integrand is the Frobenius norm of the Hessian of the scalar field, which is

$$\|\nabla^2 h\|_F^2 = h_{xx}^2 + 2h_{xy}^2 + h_{yy}^2$$

Observe that the norm of the Laplacian is

$$\|\Delta h\|^2 = (h_{xx} + h_{yy})^2 = h_{xx}^2 + 2h_{xx}h_{yy} + h_{yy}^2$$

Importantly, the difference is the mixed terms. There are two arguments to how this relates to bending energies:

We can write

$$\|\Delta h\|^2 = (h_{xx} + h_{yy})^2 = h_{xx}^2 + 2h_{xx}h_{yy} + h_{yy}^2 = \text{tr}(\nabla^2 h)^2 - 2\det(\nabla^2 h)$$

and note that  $\text{tr}(\nabla^2 h) = \Delta h$ . This means that

$$\|\Delta h\|^2 = (\Delta h)^2 - 2\det(\nabla^2 h)$$

The last term is the Gaussian curvature  $K$ , meaning that

$$\|\Delta h\|^2 = (2H)^2 - 2K$$

For a closed surface of fixed topology, the integral of the Gaussian curvature is constant by the Gauss-Bonnet theorem. For a flat domain with a boundary, however, this term becomes a boundary contribution and may lead to different natural boundary conditions. This does not affect the interior bending forces. Thus, we will only study the  $(2H)^2$  part.

Importantly, this has no consequence on the test case in which all  $y$ -derivatives disappear. This relationship was only shown for clarity in the connection between the Hessian energy and bending forces.

The variational derivative is derived for the particular test case later.

#### ASSUMPTIONS, LIMITATIONS

- Flat domain
- Small deformations

## 2.4 Isometric Bending Model

The isometric bending model (IBM) discretizes the Laplace-Beltrami operator (without restricting the domain), but assumes isometry [1]. This means that if the embedding  $\mathbf{X}$  is changed, the metric remains unchanged:

$$g = g_0$$

Therefore, we are studying the energy functional

$$E[\mathbf{X}] = \frac{\kappa}{2} \int_S \langle \Delta_0 \mathbf{X}, \Delta_0 \mathbf{X} \rangle^2 dA_0$$

where  $g_0$ ,  $dA_0$  and  $\Delta_0$  are fixed. Note that the vector Laplace-Beltrami operator is defined as the element-wise Laplace-Beltrami,

$$\Delta_0 \mathbf{X} = \begin{pmatrix} \Delta_0 X^1(u^1, u^2) \\ \Delta_0 X^2(u^1, u^2) \\ \Delta_0 X^3(u^1, u^2) \end{pmatrix}$$

Now, since we are interested in the force density, vary the embedding by some vector valued function  $\boldsymbol{\eta}$  and some small parameter  $\epsilon$ ,  $\mathbf{X} \rightarrow \mathbf{X} + \epsilon \boldsymbol{\eta}$ .

Define the variation

$$\delta = \left. \frac{d}{d\epsilon} \right|_{\epsilon=0}$$

evaluated at  $\epsilon = 0$ . Because of the fixed metric and the linearity of the Laplace-Beltrami operator,



$$\delta(\Delta_0(\mathbf{X} + \epsilon\boldsymbol{\eta})) = \Delta_0\boldsymbol{\eta}$$

and

$$\delta(dA_0) = 0$$

This gives

$$\begin{aligned} \delta E &= \left. \frac{d}{d\epsilon} E[\mathbf{X} + \epsilon\boldsymbol{\eta}] \right|_{\epsilon=0} \\ &= \frac{\kappa}{2} \int \delta(\|\Delta_0\mathbf{X} + \epsilon\Delta_0\boldsymbol{\eta}\|^2) dA_0 \\ &= \frac{\kappa}{2} \int \delta(\|\Delta_0\mathbf{X}\|^2 + 2\epsilon\Delta_0\mathbf{X} \cdot \Delta_0\boldsymbol{\eta} + \epsilon^2\|\Delta_0\boldsymbol{\eta}\|^2) dA_0 \\ &= \frac{\kappa}{2} \int [\delta(\|\Delta_0\mathbf{X}\|^2) + \delta(2\epsilon\Delta_0\mathbf{X} \cdot \Delta_0\boldsymbol{\eta}) + \delta(\epsilon^2\|\Delta_0\boldsymbol{\eta}\|^2)] dA_0 \end{aligned}$$

The three terms are

$$\delta(\|\Delta_0\mathbf{X}\|^2) = \frac{d}{d\epsilon} \|\Delta_0\mathbf{X}\|^2 = 0,$$

since it is independent of  $\epsilon$ ,

$$\delta(2\epsilon\Delta_0\mathbf{X} \cdot \Delta_0\boldsymbol{\eta}) = 2\Delta_0\mathbf{X} \cdot \Delta_0\boldsymbol{\eta},$$

and

$$\delta(\epsilon^2\|\Delta_0\boldsymbol{\eta}\|^2) = 0.$$

This gives

$$\boxed{\delta E = \kappa \int \Delta_0\mathbf{X} \cdot \Delta_0\boldsymbol{\eta} dA_0}$$

This can be simplified in different ways to get an equation for the force density, e.g. through Greens identities and more. However for this paper, the equation will only be derived for the test case relevant to the study.

### 3 Discretization

We briefly discuss how the two energies are discretized and other relevant quantities (e.g. curvature). Time complexities are also estimated, in the form  $O(N)$  for number of elementary operations  $N$ . One idea behind the discrete IBM was to introduce a new method of dynamics (away from the usual numerical differentiation of the discrete potential energy), that considered bending and stretching separately in half time steps.

An important question to be raised is this: is the goal of this new method to reduce computation time, or just to provide another ("simpler") scheme? If it is the former, we must consider properly whether the split scheme really speeds up calculations. The reason the IBM is successful in reducing computation times is that it applies to isometric bending, i.e. inextensible surfaces, meaning that the matrix  $Q$  only needs to be calculated once at the start of the simulation. If stretching is introduced,  $Q$  must be recalculated at every timestep, severely affecting computation times. This should be considered thoroughly.

#### 3.1 Flat Domain Hessian Energy

Libigl's `igl::hessian_energy` uses theory from the paper "Natural Boundary Conditions for Smoothing in Geometry Processing" [2].

Take a flat domain  $\Omega \subset \mathbb{R}^2$  with a scalar field  $u : \Omega \rightarrow \mathbb{R}$

The Hessian smoothness energy in the paper is defined as

$$E_H(u) = \frac{1}{2} \int_{\Omega} \|\nabla^2 u\|_F^2 dA$$

where  $\nabla^2$  denotes the Hessian. Since we are working on a flat domain, we can embed the surface in Cartesian coordinates which gives

$$\|\nabla^2 u\| = u_{xx}^2 + 2u_{xy}^2 + u_{yy}^2$$

The energy penalizes changes in the gradient. It therefore vanishes for linear functions. Its Euler-Lagrange equation is the biharmonic equation

$$\Delta^2 u = 0$$

The corresponding natural boundary conditions allow the solution to remain linear near unconstrained boundaries, reducing boundary artefacts.

On a triangular mesh, the scalar field is represented by its values at the  $N$  vertices,

$$\begin{pmatrix} u_1 & u_2 & \cdots & u_N \end{pmatrix}^T$$

Using linear finite-element basis functions,  $u$  is interpolated linearly over each triangle. Its gradient is therefore constant within each triangle, making its ordinary Hessian zero. The second-derivative information instead appears through changes in the gradient between neighbouring triangles ??.

The method therefore reconstructs the Hessian in a weak sense. After integration by parts, the second derivatives can be expressed using first derivatives of the piecewise-linear basis functions. In matrix form, the discrete Hessian coefficients satisfy

$$M\mathbf{h} = B\mathbf{u},$$

and hence

$$\mathbf{h} = M^{-1}B\mathbf{u}.$$

Here,  $B$  contains the discrete derivative information and  $M$  is a mass matrix used to integrate over the mesh. The vector  $\mathbf{h}$  contains the components of the reconstructed Hessian.

The discrete energy is obtained by integrating the squared Frobenius norm of this Hessian:

$$\frac{1}{2}\mathbf{h}^T M \mathbf{h}.$$

Substituting  $\mathbf{h} = M^{-1}B\mathbf{u}$  gives

$$E = \frac{1}{2}\mathbf{u}^T Q \mathbf{u},$$

where

$$Q = B^T M^{-1} B.$$

$$E_h = \frac{1}{2}h^T Q h$$

Thus, `igl::hessian_energy` uses the mesh geometry and connectivity to assemble the quadratic energy matrix  $Q$ . Once  $Q$  has been constructed, it can be applied to any vertex-based scalar field  $\mathbf{u}$ .

Now, the nodal force can be taken as the derivative of  $E$  with respect to position,  $\mathbf{u}$ . Importantly, this is only true if  $Q$  is considered constant in position, meaning that a change in coordinates does not change  $Q$ .

### 3.1.1 Time Complexity

We calculate the time complexity of computing  $Q$ . Let  $N$  be the number of vertices in the mesh. Since  $Q = B^T M^{-1} B$ , the costs of constructing  $B$ ,  $M$ ,  $M^{-1}$ , and the overall product are required.

Note that  $B = - \int \phi'_i \phi'_k dA$ , where  $\phi_j$  are the hat functions applied to each vertex. The matrix  $B$  is assembled from local contributions on each triangle. Since each triangle has only three vertices, computing its basis-function gradients and inserting its contribution into  $B$  requires a fixed amount of work. Therefore, the cost of assembling  $B$  is  $O(F)$ . For a uniform mesh, the number of faces is approximately proportional to the number of vertices since three vertices build a face. This means that  $O(F) = O(N)$ , and the time complexity of constructing  $B$  is  $O(N)$ . Importantly, each vertex interacts only with vertices in a small local neighbourhood. Therefore, each row of  $B$  contains only a fixed number of non-zero entries, meaning that the total number of entries in  $B$  is  $O(N)$ .

In `igl::hessian_energy`,  $M$  is a diagonal mass matrix. Its diagonal entries contain the areas associated with the mesh vertices. Constructing these areas requires summing local triangle contributions and therefore costs  $O(F) = O(N)$ .

Because  $M$  is diagonal, its inverse is obtained by inverting each diagonal entry separately:

$$(M^{-1})_{ii} = \frac{1}{M_{ii}}.$$

There are  $O(N)$  diagonal entries, up to a fixed number of Hessian components. Consequently,

For the multiplications, first consider

$$C = M^{-1} B.$$

Since  $M^{-1}$  is diagonal, this operation only scales each row of  $B$ . Its cost is therefore proportional to the number of non-zero entries in  $B$ , which is  $O(N)$ .

Finally, the operation  $Q = B^T C$  corresponds to multiplying two sparse matrices, each with  $O(N)$  elements. The time complexity of this operation is trickier, but if we assume that the matrix  $B$  is randomly filled (which may not be too bad of an approximation, considering that face connections may not always result in uniform or diagonal matrices), then the time complexity can be as fast as  $O(N)$  ???. With this assumption, the overall time complexity is  $O(N)$  for computing  $Q$ .

### 3.2 IBM

The quadratic isometric bending model (IBM) was introduced as an efficient approximation of curvature-based bending for surfaces where stretching is negligible [3].

Let  $\mathbf{X} \rightarrow \mathbb{R}^3$  be the embedding of a surface.

The discretized energy is precisely the curvature based bending energy discussed previously,

$$E = \frac{\kappa}{2} \int_S (2H)^2 dA$$

Particularly, using

$$\Delta_g \mathbf{X} = 2H\mathbf{n},$$

the energy that is discretized is based on the Laplace-Beltrami, i.e.

$$E = \frac{\kappa}{2} \int_{\Omega} \|\Delta_g \mathbf{X}\|^2 dA$$

The central assumption is that the surface deforms isometrically, meaning that

$$g = g_0$$

i.e. the metric remains unchanged after deformation. Since  $\Delta_g$  and  $dA$  depend only on the metric, they remain constant during an isometric deformation. The energy is therefore quadratic in the embedding coordinates  $\mathbf{X}$ .

#### 3.2.1 Finite Element Discretization

We begin discussing the discretization by introducing a finite element basis.

Let the triangulated surface have  $N$  vertices, with positions collected in the matrix (where  $\mathbf{x}_i$  are column vectors with the global coordinates)

$$\begin{pmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_N^T \end{pmatrix} \in \mathbb{R}^{N \times 3}.$$

A scalar function  $u$  is represented using finite-element basis functions  $\Phi_i$ :

$$u_h = \sum_{i=1}^N u_i \Phi_i.$$

The FEM stiffness matrix  $L$  and mass matrix  $M$  are

$$L_{ij} = \int_S \langle \nabla \Phi_i, \nabla \Phi_j \rangle, dA,$$

and

$$M_{ij} = \int_S \Phi_i \Phi_j dA.$$

The stiffness matrix  $L$  is the weak discrete Laplace-Beltrami operator. The area-normalized discrete Laplacian is

$$\Delta_g u \approx M^{-1} L \mathbf{u}.$$

Alternatively,  $M$  is a lumped mass matrix where the diagonals contain approximated areas, e.g. using Voronoi area or Barycentric area schemes.

Applying this to each coordinate function of  $\mathbf{X}$  gives the discrete mean-curvature vector:

$$\Delta_g \mathbf{X} \approx M^{-1} L \mathbf{X}.$$

The curvature-squared integral is then approximated by

$$\frac{\kappa_b}{2} (M^{-1} L \mathbf{X})^T M (M^{-1} L \mathbf{X}).$$

Assuming  $M$  is symmetric, this becomes

$$E_b = \frac{\kappa_b}{2} \mathbf{X}^T L^T M^{-1} L \mathbf{X}.$$

The quadratic bending matrix is therefore

$$Q = L^T M^{-1} L,$$

giving

$$E = \frac{\kappa_b}{2} \sum_{\alpha=1}^3 (\mathbf{X}^\alpha)^T Q \mathbf{X}^\alpha.$$

Here,  $\mathbf{X}^\alpha$  denotes one of the three coordinate vectors. The matrix  $M$  in this expression is the geometric finite-element mass matrix used to integrate over the surface.

Need to consider: discretizing using Crouzeix-Raviart vs Voronoi area (as I implemented). What are the differences? One clear difference is Voronoi is lumped, how does that differ from the Crouzeix-Raviart?

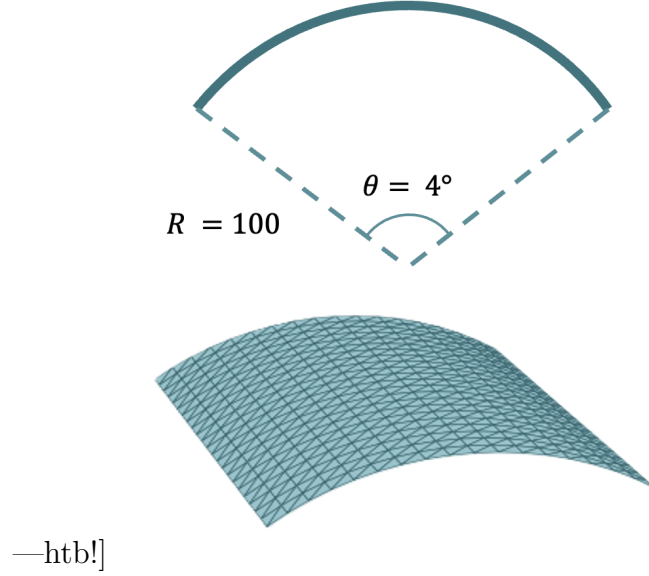


Figure 1: Exaggerated illustration of arc profile

### 3.2.2 Time Complexity

The calculation resembles the FDHE time complexity calculation and is omitted for now. Benchmark tests are ran instead.

## 4 Test Case

We study a shallow arc with radius  $R = 100$ , straight axis length  $L = 100$ , and angular length  $\theta = 4^\circ$ .

Its principal curvatures are

$$k_1 = 1/R, \quad k_2 = 0$$

giving  $H = 1/(2R) = 1/200$ , and  $K = 0$ .

The continuum force densities were derived as the variational derivatives of the respective energies for this test case. These provide the analytical values against which the numerical force densities are compared.

### 4.1 Flat Domain Hessian Energy

We will derive the variational derivative for the test case. We can embed the plane (before deformation) as

$$\mathbf{X}_0 = \begin{pmatrix} x \\ y \\ 0 \end{pmatrix}.$$

Then, the plane is deformed as

$$\mathbf{X} = \begin{pmatrix} x \\ y \\ h(x, y) \end{pmatrix}.$$

where  $h$  is the height field of the circular profile. For a circular profile with radius  $R$  and length (in the  $y$ -direction)  $L$ , we get

$$h(x) = \sqrt{R^2 - \left(\frac{L}{2}\right)^2} + \sqrt{R^2 - x^2}$$

We can now use the multivariable Euler-Lagrange equation to get an equation for the *interior force density* for the test case (ignoring boundary terms, since we only calculate interior forces)

$$\text{Let } \mathcal{L}(h_{,xx}, h_{,yy}) = \frac{\kappa}{2}(\Delta h)^2 = \frac{\kappa}{2}\left(\frac{\partial^2 h}{\partial x^2} + \frac{\partial^2 h}{\partial y^2}\right)^2$$

Euler-Lagrange's equation

$$\frac{\delta E}{\delta h} = \frac{\partial \mathcal{L}}{\partial h} - \sum \frac{\partial}{\partial x^i} \left( \frac{\partial \mathcal{L}}{\partial h_{,i}} \right) + \sum \frac{\partial^2}{\partial x^i \partial x^j} \left( \frac{\partial \mathcal{L}}{\partial h_{,ij}} \right)$$

$$\text{where } \partial h_{,ij} = \frac{\partial^2 h}{\partial x^i \partial x^j}$$

Since  $\mathcal{L}$  only depends on the second derivatives, and only  $h_{xx}$  is nonzero, this reduces to

$$\frac{\delta E}{\delta h} = \kappa h_{xxxx}$$

(More generally, this is the biharmonic operator acting on  $h$ .)

This then gives the force density

$$f = -\kappa h_{xxxx} = -\frac{3\kappa R^2(R^2 + 4x^2)}{(R^2 - x^2)^{7/2}}$$

Importantly, we must ask what direction this force density is. The answer is: in the  $\mathbf{e}_3$  direction, i.e. the vertical direction of the slope. In the small deformations assumption, this is approximately the normal direction.

Then, for  $R \gg x$ , we get that



$$\mathbf{f} \approx -\frac{3\kappa}{R^3} \mathbf{e}_3$$

Thus, the analytical force density in the  $\mathbf{e}_3$  direction has magnitude  $3\kappa/R^3$ .

## 4.2 IBM

We study a shallow circular arc with radius  $R$  and a given angular length. Previously, we defined a plane embedded in  $\mathbb{R}^3$  as

$$\mathbf{X}_0 = \begin{pmatrix} x \\ y \\ 0 \end{pmatrix}.$$

We then deformed the plane using the embedding

$$\mathbf{X} = \begin{pmatrix} x \\ y \\ h(x, y) \end{pmatrix}.$$

This deformation is not isometric. The metric of the original plane is

$$g_0 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix},$$

whereas the metric of the deformed surface is

$$g = \begin{pmatrix} 1 + h_x^2 & h_x h_y \\ h_x h_y & 1 + h_y^2 \end{pmatrix}.$$

Therefore,

$$g \neq g_0.$$

However, for small deformations,

$$h_x^2, h_y^2, h_x h_y \approx 0,$$

so the deformation is approximately isometric.

Isometry is important for the discrete IBM. We therefore have two possibilities:

1. Since the test case involves small deformations, use the embedding

$$\mathbf{X} = \begin{pmatrix} x \\ y \\ h(x, y) \end{pmatrix}, \quad h(x, y) = \sqrt{R^2 - x^2} + C,$$

where  $C$  is a constant.

2. Introduce an arc-length parametrization, which is exactly isometric.

We use the second approach for the numerical study because we wish to consider the models in their intended settings.

If  $s$  is the arc length, then

$$s = R\theta, \quad \theta = \frac{s}{R}.$$

The Cartesian coordinates of the arc are

$$x = R \sin \theta, \quad z = R(1 - \cos \theta).$$

Therefore, the deformed embedding is

$$\mathbf{X}(s, y) = \begin{pmatrix} R \sin(s/R) \\ y \\ R(1 - \cos(s/R)) \end{pmatrix}.$$

This is a circle of radius  $R$ , shifted so that  $z = 0$  when  $s = 0$ .

The tangent vectors are

$$\mathbf{X}_s = \begin{pmatrix} \cos(s/R) \\ 0 \\ \sin(s/R) \end{pmatrix}, \quad \mathbf{X}_y = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}.$$

Hence,

$$g = \begin{pmatrix} \mathbf{X}_s \cdot \mathbf{X}_s & \mathbf{X}_s \cdot \mathbf{X}_y \\ \mathbf{X}_y \cdot \mathbf{X}_s & \mathbf{X}_y \cdot \mathbf{X}_y \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = g_0.$$

Thus, the embedding is isometric.

Since  $g_{ij}$  is the identity matrix,

$$\sqrt{g} = 1, \quad g^{ij} = g_{ij},$$

and the Laplace-Beltrami operator reduces to

$$\Delta_0 = \frac{\partial^2}{\partial s^2} + \frac{\partial^2}{\partial y^2}.$$

Applying this to the embedding gives

$$\Delta_0 \mathbf{X} = \frac{1}{R} \begin{pmatrix} -\sin(s/R) \\ 0 \\ \cos(s/R) \end{pmatrix}.$$

Now let

$$\boldsymbol{\eta} = \begin{pmatrix} \eta^1 \\ \eta^2 \\ \eta^3 \end{pmatrix}.$$

Using

$$\delta E = \kappa \int_{\Omega} \Delta_0 \mathbf{X} \cdot \Delta_0 \boldsymbol{\eta} dA_0,$$

we obtain

$$\delta E = \frac{\kappa}{R} \int_{\Omega} [ -(\eta_{ss}^1 + \eta_{yy}^1) \sin(s/R) + (\eta_{ss}^3 + \eta_{yy}^3) \cos(s/R) ] ds dy.$$

For the rectangular parameter domain

$$\Omega = [s_0, s_1] \times [y_0, y_1],$$

this becomes

$$\delta E = \frac{\kappa}{R} \int_{y_0}^{y_1} \int_{s_0}^{s_1} [ (\eta_{ss}^3 + \eta_{yy}^3) \cos(s/R) - (\eta_{ss}^1 + \eta_{yy}^1) \sin(s/R) ] ds dy.$$

Integrating by parts twice and ignoring the boundary terms, since we are concerned with the interior force, gives

$$\delta E = \frac{\kappa}{R^3} \int_{\Omega} [ \eta^1 \sin(s/R) - \eta^3 \cos(s/R) ] dA_0.$$

The integrand can be written as

$$\frac{\kappa}{R^3} \begin{pmatrix} \sin(s/R) \\ 0 \\ -\cos(s/R) \end{pmatrix} \cdot \boldsymbol{\eta}.$$

The energy  $E[\mathbf{X}]$  takes an entire embedding  $\mathbf{X}$  and returns a scalar. This is analogous to the multivariable result

$$E : \mathbb{R}^n \rightarrow \mathbb{R},$$

for which

$$\left. \frac{d}{d\epsilon} E(\mathbf{x} + \epsilon \mathbf{v}) \right|_{\epsilon=0} = \nabla E(\mathbf{x}) \cdot \mathbf{v} = \sum_i \frac{\partial E}{\partial x_i} v_i.$$

For the field  $\mathbf{X}$ , there is one degree of freedom at every point  $(s, y)$ . Therefore, the sum becomes an integral:

$$\delta E = \int_{\Omega} \frac{\delta E}{\delta \mathbf{X}} \cdot \boldsymbol{\eta} dA_0.$$

The bending force density is defined as

$$\mathbf{f}_b = -\frac{\delta E}{\delta \mathbf{X}}.$$

It follows that

$$\mathbf{f}_b = \frac{\kappa}{R^3} \begin{pmatrix} -\sin(s/R) \\ 0 \\ \cos(s/R) \end{pmatrix}.$$

The unit normal is

$$\mathbf{n} = \mathbf{X}_s \times \mathbf{X}_y = \begin{pmatrix} \cos(s/R) \\ 0 \\ \sin(s/R) \end{pmatrix} \times \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} -\sin(s/R) \\ 0 \\ \cos(s/R) \end{pmatrix}.$$

Thus,

$$\boxed{\mathbf{f}_b = \frac{\kappa}{R^3} \mathbf{n}}$$

It follows that the mean curvature of the test shape is given by

$$\Delta_0 \mathbf{X} = 2H\mathbf{n}$$

Since  $\Delta_0 \mathbf{X} = \frac{1}{R}\mathbf{n}$ , it follows that  $2H = \frac{1}{R}$ .

With a radius  $R = 100$ , the mean curvature  $H = \frac{1}{200}$ .

## 5 Method

### 5.1 Mesh construction

The mesh is constructed using a uniform grid (which gives a more even average mesh spacing compared to the random grid). The length along the straight axis  $L_z = 100$ , and the radius of the circular profile is defined as  $R = 100$ . Note that the curvature of the circular profile is then  $1/R$ . The angular length in degrees of the arc is also supplied through `theta_min` and `theta_max`. The number of points along the arc,  $n_\theta$  is chosen and refined (i.e. increased) throughout the simulation. The number of points along the straight axis,  $n_y$ , is calculated to ensure that the triangles are as isosceles as possible. This is done using the formula

$$n_y = \text{round} \left( \frac{L}{R\Delta\theta} (n_\theta - 1) \right) + 1$$

Note that  $n_\theta$  is the number of points, which is why one is subtracted from it to get the number of intervals. Similarly, one is added to  $n_z$  to convert from intervals to points.

For functionality with `libigl`, both the vertex coordinates (which are easy to calculate, as both  $x, y$  are supplied and  $z$  is the height, calculated using trigonometry), and the face connections are required. This is essentially where the triangle faces are defined. This is done by iterating through every  $z$ -row, each containing  $n_z$  points, and defining them to a face. The third point forming that face is taken as the corresponding point on the next row. By iterating through all the points, these face connections are stored in an array.

### 5.2 Numerical Force Density

After the mesh is created (as a set of points and face connections), the forces can then be calculated using `Libigl`. Importantly, the theoretical forces are in fact force densities, whereas the discrete models supply forces. To convert the forces into force densities, triangle areas are used. Since the forces are calculated at each vertex, the Voronoi area around that point is taken and the force is divided by that area to get a

local force density. The Voronoi area is calculated by taking the area of the polygon whose vertices are at the center of each surrounding triangle. Note that this takes into account bends along the meshes, making it a good choice for operators such as the Laplace-Beltrami, which measure bends and smoothness.

The force densities and curvatures are computed at every vertex for 8 different meshes, with  $n_\theta$  ranging from

| $n_\theta$ | $n_y$ | Total points $n_\theta n_y$ |
|------------|-------|-----------------------------|
| 6          | 73    | 438                         |
| 8          | 101   | 808                         |
| 12         | 159   | 1 908                       |
| 18         | 245   | 4 410                       |
| 24         | 330   | 7 920                       |
| 32         | 445   | 14 240                      |
| 40         | 560   | 22 400                      |
| 48         | 674   | 32 352                      |

Table 1: Mesh resolutions for  $L = 100$ ,  $R = 100$ , and  $\Delta\theta = 4^\circ$ .

Since the analytical expressions describe the *interior* Euler–Lagrange force, boundary contributions were excluded by evaluating errors only on vertices sufficiently far from the patch boundary. The RMS error is taken and associated with that number of points. Plots are then presented using average mesh spacing, which is taken as the surface area divided by the total number of points. This is so that the results are more broadly applicable to general meshes, e.g. random meshes.

**A note for the discrete IBM:**

Since the metric is frozen, the original plane should be sent in and the new coordinates (i.e. change in height field) should be taken to calculate the force.

Small deformations mean this may not matter, but it is better to be careful.

### 5.3 Mean Curvature Convergence

Mean curvature convergence is tested on a unit sphere. The meshes are created using the Python library `meshzoo`, and the data is plotted against the *average mesh spacing*  $a_n$  which is taken as the total surface area (which is  $4\pi$  for the unit sphere) divided by the total number of vertices.

The expected mean curvature is  $1/R$ , and the experiment is ran for the following resolutions,

| $n$ | Total points |
|-----|--------------|
| 6   | 1086         |
| 8   | 1926         |
| 12  | 4326         |
| 18  | 9726         |
| 24  | 17286        |
| 32  | 30726        |
| 40  | 48006        |
| 48  | 69126        |

Table 2: Mesh resolutions for unit sphere

## 5.4 Time Benchmark

Lastly, time measurements are taken for the construction time for  $Q$  using both methods. A total of ten trials are run and the average time across all these trials are taken. Furthermore, computation time for the forces are also computed (corresponding to a matrix-vector multiplication).

# 6 Results

## 6.1 FDHE

I made a very peculiar observation. For a somewhat fine mesh, if the matrix  $Q$  is constructed with the arc geometry, the force density matches the theoretical value for the flat domain HE.

On the contrary, if  $Q$  is constructed with the plane geometry, the force density matches with the theoretical IBM value! I believe this is a result of the small deformations assumption, since the bending is approximately isometric. But why is this consistent? Is something wrong with the analytical value?

**The problem:** Since the arc mesh was constructed from an isometric embedding, some  $x$  and  $y$  coordinates shifted. This meant that the corresponding  $z$ -values were at the shifted coordinates. Applying these values to the FDHE meant calculating the forces on a cosine profile, not a circular profile. This was fixed by taking the exact height field shifted values as the new  $z$ -coordinates, which gave the expected nodal force densities.

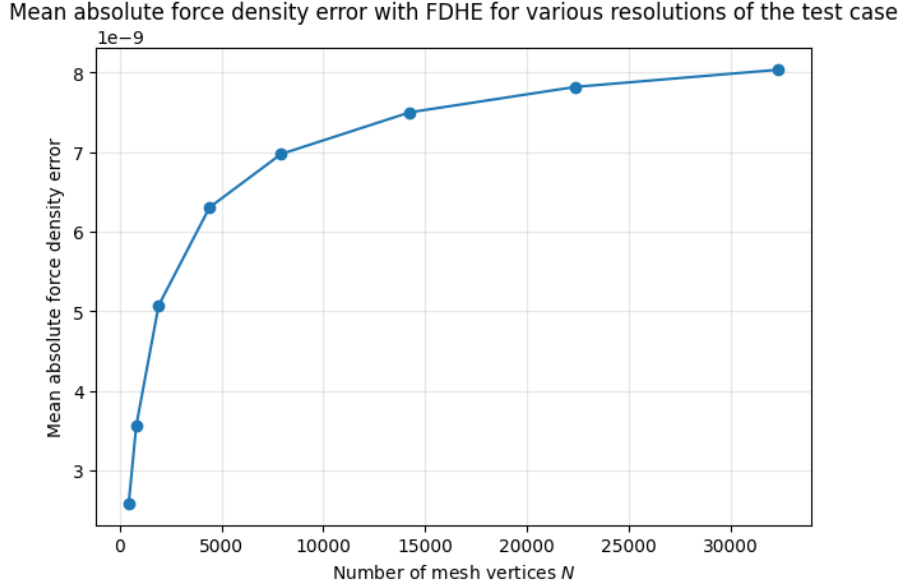


Figure 2: Mean force density error with FDHE method on test case with various mesh resolutions

## 6.2 IBM

### 6.2.1 Test Case

After excluding two boundary layers, the mean absolute error across all tested mesh resolutions remained below  $10^{-12}$  for curvature and  $3.5 \cdot 10^{-10}$  for normal force density, corresponding to relative errors below  $2 \cdot 10^{-8}\%$  and  $0.035\%$ , respectively.

### 6.2.2 Boundary Effects Test

Possible boundary effects are tested by masking boundary layers from the normal FD graphs.



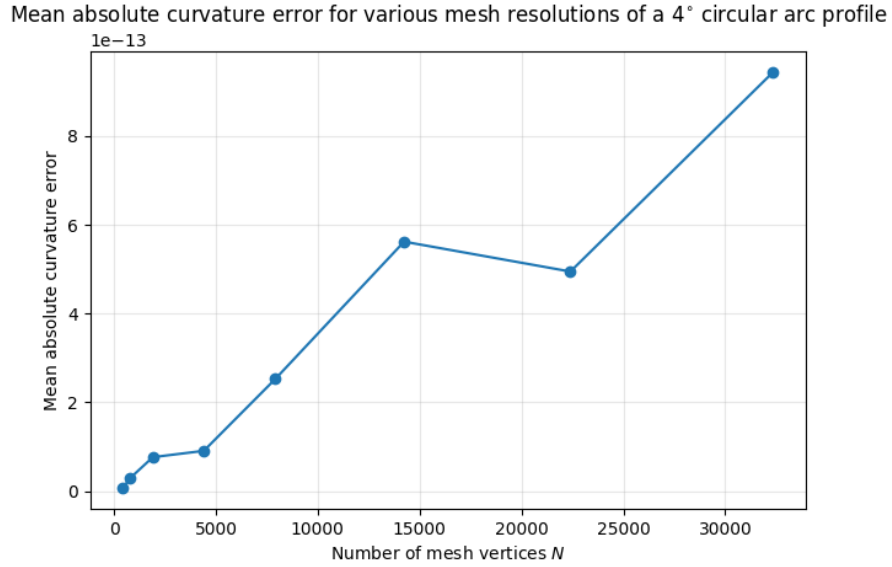


Figure 3: Mean curvature error for test case using the discrete IBM for with various mesh resolutions

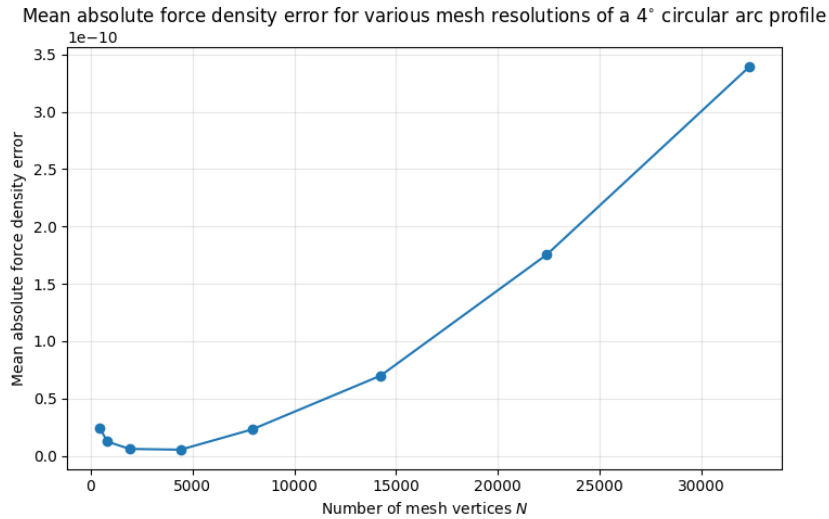
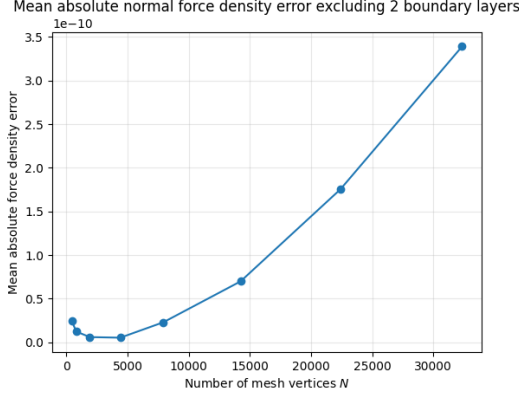


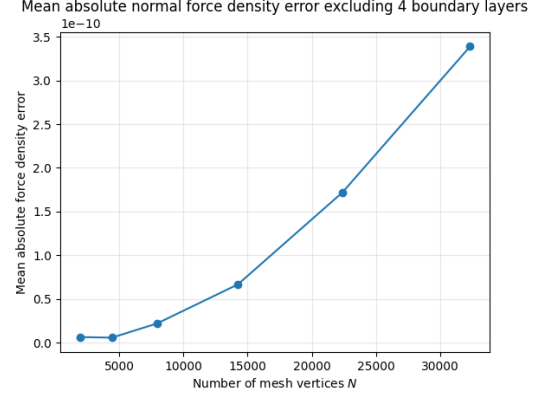
Figure 4: Mean normal force density error for test case using the discrete IBM for with various mesh resolutions

### 6.2.3 Mean Curvature Convergence

To test convergence of mean curvature, we study the mean curvatures on an icosphere that is gradually refined. The analytical mean curvature is  $1/R$ , and convergence towards this value is studied by looking at the average mean curvature across coarse to fine meshes.



(a) Two boundary layers removed.



(b) Four boundary layers removed.

Figure 5: Mean normal force-density error after removing different numbers of boundary layers.

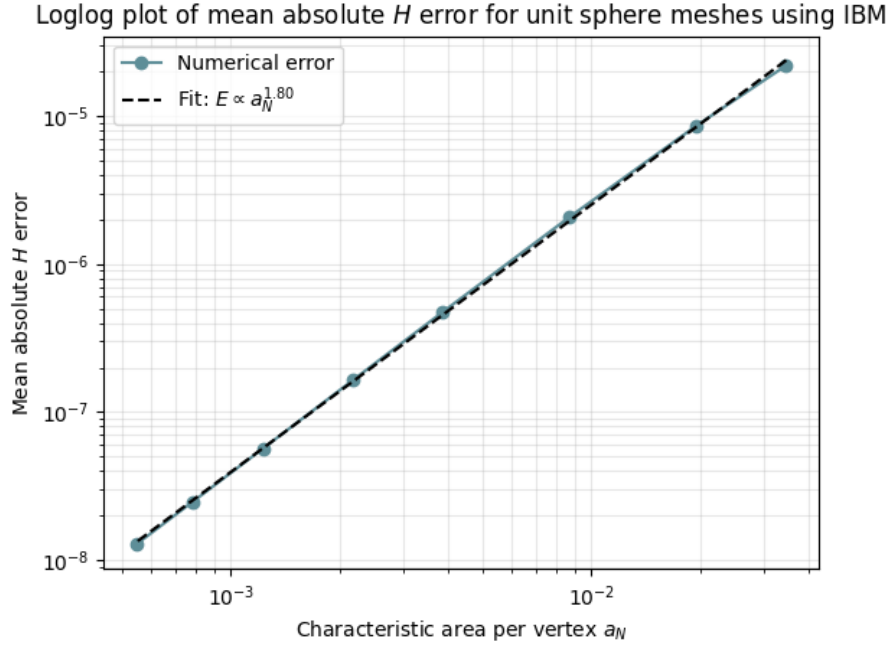


Figure 6: Loglog plot of mean curvature error for various area spacings  $a_N$

The plot of the mean average error versus average area per vertex is linear in a loglog plot. What does this mean? Mathematically, if  $E_H$  denotes the mean absolute error

$$\log E_H = C_1 p \log a_N$$

for some constant  $C_1$ . This means that

$$E_H = Ca_N^p$$

where  $C = e^{C_1}$ . The value  $p$  is measured as 1.8.

If instead one wishes to observe convergence at a length scale, the average mesh spacing can be taken as the square root of  $a_N$ . If  $a_N = h^2$ , the convergence order is given by

$$\log E_H = C_1 p \log h^2 \Rightarrow E_H \propto h^{2p}$$

### 6.3 Time Benchmark

| Number of vertices $N$ | Flat Domain HE        |                   | IBM                   |                   |
|------------------------|-----------------------|-------------------|-----------------------|-------------------|
|                        | $Q$ construction (ms) | Force ( $\mu s$ ) | $Q$ construction (ms) | Force ( $\mu s$ ) |
| 438                    | 24.824                | 136.758           | 8.947                 | 411.616           |
| 1908                   | 102.102               | 672.927           | 31.878                | 1912.94           |
| 7920                   | 430.369               | 2668.59           | 136.379               | 7961.65           |
| 32352                  | 1857.6                | 11194.8           | 556.647               | 34048.5           |

Table 3: Mean computation times over ten trials for constructing  $Q$  and evaluating the force.

An important note: although the force computation times appear to be greater for the IBM, it's important to note that the IBM applies  $Q$  to all three coordinates, where as the flat domain HE only applies  $Q$  to the height coordinates. This means that the IBM is doing three times the work of the flat domain HE for the force. For comparison, dividing the force value by three gives almost identical time complexity for computing the force. This is simply the time complexity for the matrix-vector multiplication, so this is not so interesting.

## 7 Discussion

### 7.1 Floating Point Round-off Errors

The errors did not decrease monotonically during mesh refinement for both the mean curvature and normal force density. The errors remained small, below  $10^{-11}$  for curvature and  $10^{-9}$  for normal force density. The observed increase for the finer meshes

could be caused by amplification of floating point errors, rather than divergence of the discretization. As the mesh is refined, vertex areas decrease and the entries of the *inverse mass matrix*  $M^{-1}$  increase. Consequently, small round off and cancellation errors in the discrete Laplacian can be amplified. This effect is expected to be stronger for the force density, since its calculation applies the discrete Laplacian twice and subsequently divides the nodal force by the (smaller) vertex area.

The increasing error at high resolutions persisted when additional boundary layers were excluded as shown in figure 5, indicating that it was not caused by boundary effects. Since the errors remained small and the inverse mass matrices increasingly amplify floating point errors as the vertex areas decrease, the trend is most likely due to precision limits.

## 7.2 Mean Curvature Convergence

The icosphere test results indicate near fourth order convergence in mesh spacing (where a smaller mesh spacing indicates a finer mesh).

## 7.3 Time complexity

Both methods show linear time complexity in the number of vertices,  $N$ . However, the IBM constructs  $Q$  faster (by a factor of around 3). The computation time for the forces reflects the time complexity of matrix-vector multiplication in Eigen, and is roughly  $O(N)$  for these sample sizes.

## 7.4 Applications

[Improve + source] There are various applications to this study. Primarily the IBM can be applied to thin shell isometric deformations. Cloth dynamics is a prime example of this, where the stretching is negligible.

\* Another interesting application, which is the broader goal with this project, is applications to simulating fluid membranes. This application is trickier, as fluid membranes are characterized by stretching and shifting geometries. While the IBM enforces precisely the opposite: a fixed geometry (i.e. fixed neighbour lists, etc). However, typical methods that involve differentiating the  *Helfrich hamiltonian*  are costly. Future studies may want to test whether the IBM can be implemented, and whether fluid membrane dynamics can be split into independent stretching and bending steps. On the other hand, this requires recomputing  $Q$  for each time step. Whether this implementation is faster requires further study.

## 8 Conclusion

This study compares two discrete models of surface bending, the FDHE and IBM models. Error convergence for the Laplacian-based curvature is studied using icospheres. The results indicate that both methods are highly accurate, with the IBM method indicating a relative error of 0.035%. The time complexities for computing the discrete operators was linear for both methods.

## References

- [1] Miklós Bergou et al. “A Quadratic Bending Model for Inextensible Surfaces”. In: *Proceedings of the Fourth Eurographics Symposium on Geometry Processing*. SGP '06. Eurographics Association, 2006, pp. 227–230. DOI: 10.5555/1281957.1281987. URL: <https://www.cs.columbia.edu/cg/pdfs/9-QuadBend.pdf>.
- [2] Oded Stein et al. *Natural Boundary Conditions for Smoothing in Geometry Processing*. arXiv:1707.04348 [cs.GR]. July 2017. DOI: 10.48550/arXiv.1707.04348. URL: <http://arxiv.org/abs/1707.04348> (visited on 06/05/2026).
- [3] Max Wardetzky et al. “Discrete quadratic curvature energies”. en. In: *Computer Aided Geometric Design* 24.8-9 (Nov. 2007), pp. 499–518. ISSN: 01678396. DOI: 10.1016/j.cagd.2007.07.006. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0167839607000891> (visited on 06/05/2026).